

InsightBoard

Informe Completo del Proyecto

Fecha: 08/09/2026

Version: 1.0.0

Autor: LillianaU

Plataforma de analitica con recoleccion de datos, reportes personalizados y visualizaciones interactivas
construida con Django REST Framework.

Indice

1. Resumen ejecutivo
2. Arquitectura del sistema
3. Stack tecnologico
4. Estructura del proyecto
5. Capas de seguridad
6. Endpoints de la API
7. Modelos de base de datos
8. Frontend
9. Testing
10. Despliegue
11. Credenciales de prueba
12. Tutoriales (C0-C9)

1. Resumen ejecutivo

InsightBoard es una plataforma de analitica web que permite recolectar eventos, generar metricas automaticas, crear reportes personalizados y visualizar datos interactivos con graficos Chart.js. El proyecto esta construido con Django 6.1 y Django REST Framework, desplegado en Render con PostgreSQL.

2. Arquitectura del sistema

El sistema sigue una arquitectura cliente-servidor con API REST:

Componente	Tecnologia	Funcion
Frontend	HTML + Bootstrap + Chart.js	Interfaz de usuario
Backend	Django 6.1 + DRF	API REST y logica
Base de datos	PostgreSQL 17	Almacenamiento
Cache/Tasks	Redis + Celery	Tareas en segundo plano
Auth	JWT (SimpleJWT)	Autenticacion segura
Deploy	Render (Free Tier)	Hosting en la nube

3. Stack tecnologico

Capa	Tecnologia	Version
Backend	Django	6.1.1
API REST	Django REST Framework	3.18.0
Documentacion	drf-spectacular (Swagger)	0.30.0
Auth	django-rest-framework-simplejwt	5.5.1
CORS	django-cors-headers	4.9.0
Base de datos	psycpg2-binary	2.9.12
Cache	redis	8.1.0
Tareas	celery	5.6.3
Frontend	Bootstrap 5 + Chart.js	-
Container	Docker + Docker Compose	-
Servidor	gunicorn	26.2.0
Testing	pytest + pytest-django	9.1.1
Reportes PDF	reportlab	5.0.1
Excel	openpyxl	3.1.5

4. Estructura del proyecto

InsightBoard/

- config/ # Configuración de Django
- ■■■■ settings/ # base.py, development.py, production.py
- ■■■■ urls.py # Rutas de la API
- ■■■■ celery_app.py # Tareas en segundo plano
- ■■■■ wsgi.py / asgi.py # Servidores web
- apps/ # Aplicaciones Django
- ■■■■ analytics/ # Modelos, views, serializers, services
- ■■■■ users/ # Autenticación y usuarios
- ■■■■ core/ # Utilidades generales
- frontend/ # Páginas web
- ■■■■ login.html # Página de login
- ■■■■ index.html # Dashboard principal
- ■■■■ events.html # Registro de eventos
- ■■■■ reports.html # Reportes personalizados
- ■■■■ css/style.css # Estilos
- ■■■■ js/auth.js, app.js # JavaScript
- tutorials/ # Libro de tutoriales (C0-C9)
- Dockerfile # Imagen Docker
- docker-compose.yml # Servicios Docker
- pyproject.toml # Dependencias Python
- conftest.py # Configuración pytest
- manage.py # Comando de gestión

5. Capas de seguridad (8)

#	Capa	Descripcion
1	SECRET_KEY	Nunca hardcodeada, obligatoria via variable de entorno
2	DEBUG	Por defecto False en produccion
3	ALLOWED_HOSTS	Solo dominios explicitos configurados
4	CORS	Solo origenes permitidos (no permite todos)
5	Rate limiting	30/hora anon, 1000/hora autenticado
6	JWT Seguro	Tokens con rotacion y blacklist
7	XSS Protection	Frontend usa textContent en vez de innerHTML
8	SQL Injection	Django ORM previene inyeccion SQL

6. Endpoints de la API

Metodo	Endpoint	Auth
POST	/api/auth/register/	No
POST	/api/auth/login/	No
GET	/api/auth/me/	JWT
POST	/api/events/ingest/	No
POST	/api/events/csv/	No
GET	/api/events/list/	JWT
GET/POST	/api/events/sources/	JWT
GET	/api/metrics/summary/	JWT
GET	/api/metrics/timeseries/	JWT
GET	/api/metrics/breakdown/	JWT
GET	/api/metrics/heatmap/	JWT
POST	/api/metrics/refresh/	JWT
GET/POST	/api/reports/	JWT
GET/PUT/DELETE	/api/reports/<id>/	JWT
GET	/api/docs/	No
GET	/api/health/	No

7. Modelos de base de datos

User (apps.users)

- email (EmailField, unique, PK via USERNAME_FIELD)
- username (CharField)
- is_staff, is_superuser (BooleanField)

DataSource (apps.analytics)

- name (CharField)
- description (TextField)
- created_by (FK -> User, nullable)
- created_at (DateTimeField, auto)

Event (apps.analytics)

- source (FK -> DataSource)
- event_type (CharField)
- payload (JSONField)
- user (FK -> User, nullable)
- created_at (DateTimeField, auto)

DailyMetric (apps.analytics)

- date (DateField)
- event_type (CharField)
- count (BigIntegerField)
- unique_users (BigIntegerField)

SavedReport (apps.analytics)

- user (FK -> User)
- name (CharField)
- config (JSONField)
- created_at (DateTimeField, auto)

8. Frontend

Pagina	URL	Funcion
Login	/login.html	Iniciar sesion con JWT
Dashboard	/index.html	Estadisticas y graficos interactivos
Eventos	/events.html	Registrar y listar eventos
Reportes	/reports.html	Crear reportes personalizados
API Docs	/api/docs/	Documentacion Swagger
Admin	/admin/	Panel de administracion Django

Seguridad XSS: Todo el frontend usa `textContent` en vez de `innerHTML` para evitar ataques de inyeccion de scripts. Las funciones `escapeHtml()` sanitizan cualquier dato dinamico.

9. Testing

El proyecto incluye pruebas automatizadas con `pytest` y `pytest-django`:

Archivo	Que prueba	Tests
<code>apps/users/tests.py</code>	Modelo User (crear, super, unico)	4
<code>apps/analytics/tests.py</code>	DataSource, Event, DailyMetric, SavedReport	10
<code>apps/analytics/api_tests.py</code>	Register, Login, Events, Metrics (API)	9
TOTAL		23

Ejecutar pruebas:

```
uv run pytest -v
```

10. Despliegue

Produccion: Render (Free Tier)

URL: <https://insightboard-gf27.onrender.com/>

Local con Docker:

1. `git clone https://github.com/LillianaU/InsightBoard.git`
2. `cd InsightBoard`
3. `cp .env.example .env`
4. `docker-compose up -d`
5. `docker-compose exec web uv run python manage.py migrate`
6. `docker-compose exec web uv run python manage.py createsuperuser`

Local sin Docker:

1. `uv venv && uv sync`

2. Crear PostgreSQL: CREATE DATABASE insightboard;
3. cp .env.example .env (editar DATABASE_URL)
4. uv run python manage.py migrate
5. uv run python manage.py createsuperuser
6. uv run python manage.py runserver

11. Credenciales de prueba

Usuario	Contraseña	Descripcion
admin@insightboard.com	admin123	Administrador del sistema

12. Tutoriales (C0-C9)

El proyecto incluye un libro completo de 10 capítulos que enseña a construir InsightBoard desde cero:

Cap	Titulo	Tiempo
C0	Que es InsightBoard	10 min
C1	Preparar tu computadora	20 min
C2	Docker y base de datos	25 min
C3	Crear el proyecto Django	35 min
C4	Modelos de base de datos	30 min
C5	API REST con DRF	40 min
C6	Frontend y dashboards	35 min
C7	Autenticacion JWT	25 min
C8	Probar el proyecto	20 min
C9	Publicar en Render	30 min
	TOTAL	~5 horas

Informe generado automaticamente el 08/09/2026 01:42

Repositorio: <https://github.com/LillianaU/InsightBoard>